

===== Spring Boot Backup Videos =====

ðŸŒ™ Day - 1 : youtu.be/ye9CmNL5GjI
ðŸŒ™ Day - 2 : youtu.be/EM5XvMaKnto
ðŸŒ™ Day - 3 : youtu.be/ZCZ5DbyTai4
ðŸŒ™ Day - 4 : youtu.be/FdM1wUAeMEU

=====

Whatsapp Group for class updates : <https://chat.whatsapp.com/ItWX7Dxk9FmIkVdqIGkrsA>

=====

Spring Boot & Microservices (33 - SBMS)

=====

Trainer : Mr. Ashok

IT Exp : 10+ Yrs Exp

Skill : Java Fullstack Developer

Role : Tech Lead (Product Based Company - Hyd)

7+ Years Exp in Teaching

Started Ashok IT in 2020

Pre-Requisites :

- => Core Java
- => Adv Java (JDBC + Servlets)
- => SQL Queries

Who should learn This SBMS :

- => Freshers
- => Experienced ppl
- => For Every java developer SBMS is mandatory

Note : Without keeping SBMS in your resume getting interview calls from companies is very difficult.

=====

Course Content

=====

- 1) Spring Framework Overview
- 2) Spring Core Module (IOC & DI)
- 3) Spring Boot
- 4) Spring JDBC
- 5) Spring Data JPA (ORM)
- 6) Spring Web MVC
- 7) REST APIs (Spring REST)
- 8) Microservices
- 9) Spring Cloud
- 10) Spring Security
- 11) Spring Batch
- 12) Apache Kafka + Redis + Docker + Junit + Logging + Cloud Deployment
- 13) Mini Project Development
- 14) Fullstack Integration (Spring Boot with NG and React JS integrations)

=====

Course Info

=====

Trainer : Mr. Ashok

Course Code : 33-SBMS

Duration : 4 Months

Class Timings : 7 AM - 8:30 AM (IST) (Mon-Sat)

Class Mode : Online (zoom)

=> Daily class notes will be shared in our portal

=====

What is Fullstack Development ?

=====

Fullstack = Frontend + Backend + Database

Frontend : It contains User interface (web pages)

Backend : It contains business logic

Database : To store data permanently (Storage) => CRUD

=====

Frontend Technologies

=====

- => HTML & CSS
- => Java Script
- => Bootstrap
- => Angular / React JS

=====

Backend Technologies

=====

- => Java
- => Python
- => .Net
- => Node JS
- => PHP

=====

Databases

=====

- => SQL Databases (Oracle, MySQL, Postgres...)
- => NoSQL Databases (Mongo DB, Dynamo DB, Casandra etc...)

=====

Monolith Vs Microservices

=====

Monolith : Developing all functionalities in single project

=> Drawbacks of Monolith Architecture

- 1) Maintenance
- 2) Single Point of failure
- 3) Re-Deployment of whole project
- 4) Burden on server

Microservices : Microservices is an Architectural Design Pattern to develop application in multiple API's format.

(independently deployable services)

Hotels Booking : Hotels-api

Flights Booking : Flights-api

Cabs Booking : Cabs-api

Trains Booking : Trains-api

- 1) Easy Maintenance
- 2) Faster Development
- 3) Quick Deployments
- 4) Faster Releases
- 5) Technology independency

=> We are going to learn Microservices development using Spring Boot (Backend Development).

=====
Programming Language vs Technology Vs Framework
=====

=====
What is a Programming Language?
=====

-> Programming Languages provides set of syntaxes to develop the programs

-> Using programs we can communicate with computers

P.L
human -----> computer

-> We have several programming languages in the market

Ex: C, C++, Java, Python, C# etc.....

-> Language provides FOP, Syntaxes, Rules

=====

What is Technology ?

=====

-> Technologies are developed based on programming languages

Ex: J2EE technology is developed based on Java Language.

-> Using J2EE technology we can develop web based applications

Ex: JDBC, Servlets, JSP

Note: JDBC + Servlets + JSP technologies developed using Java language.

=====

What is Framework ?

=====

-> Framework is a semi developed software

-> Framework provides common logics for applications development

Ex:

- 1) Connect with database
- 2) Setup Connection Pool
- 3) Capture form data
- 4) Form validations

-> Using frameworks we can develop projects quickly because common logics will be provided by frameworks and we need to focus only on business logics development.

-> Using framework we can do more work in less time.

Note: Frameworks developed on top of Technologies.

=====

Types of Frameworks

=====

- 1) Frontend Frameworks : Angular, React JS (js lib)
- 2) ORM Frameworks : Hibernate
- 3) Web Framework : Struts (Outdated)
- 4) Application Framework : Spring

=====

Layered Architecture

=====

- 1) Presentation Layer
- 2) Web Layer
- 3) Service Layer
- 4) DAO Layer

=====

What is Spring ?

=====

- > Spring is a free & open source framework
- > Spring framework providing support to develop end to end applications hence it is called as Application Development framework.
- > Spring came into market in 2003
- > Spring framework developed by Interface21 and now it acquired by VMware.
- > The current version of spring is 6.x version
- > Spring framework developed in modular fasion
- > We have several modules in Spring framework like below

- 1) Spring Core
- 2) Spring Context
- 3) Spring AOP
- 4) Spring JDBC/DAO
- 5) Spring ORM
- 6) Spring Data
- 7) Spring Web MVC
- 8) Spring REST
- 9) Spring Cloud
- 10) Spring Batch
- 11) Spring Security etc...

=> Based on requirement we can use any module which is available in Spring framework.

=====

Spring Core

=====

- > It is base module of Spring framework

-> All the other modules of Spring developed based on Spring Core

-> It is providing fundamental concepts of Spring framework

- 1) IOC (Inversion of control)

- 2) DI (Dependency Injection)

- 3) Autowiring

Note: IOC & DI will help us in developing loosely coupled applications

=====

Spring Context

=====

-> It provides configuration support required for application

- 1) XML config support (Outdated)

- 2) Annotation Config support (Latest)

=====

Spring AOP

=====

-> AOP stands for Aspect oriented programming

-> It is used to separate cross cutting logics with business logics

Cross Cutting Logics : logging, security, transaction, auditing etc...

business logic : amt-credit, amt-debit, bal-check, applyLoan etc....

=====

Spring JDBC / Spring DAO

=====

-> It is used to develop Persistence Layer (DB Communication Logic)

-> Spring JDBC developed on top of JDBC API.

-> JDBC API boiler plate can be avoided using Spring JDBC module.

Note: We can directly execute SQL queries using Spring JDBC module

(no need of => loading driver, getting connection, creating stmt and closing connections)

=====

Spring ORM

=====

-> It is used to develop Persistence layer with ORM features

ORM -> Object Relational Mapping

-> Spring ORM developed on top of Hibernate framework

=====

Spring Data JPA

=====

-> It is used to develop Persistence layer with less coding

-> It is simplifying CRUD operations in DAO layer

=====

Spring Web MVC

=====

-> It is used to develop Web applications

-> We can develop Web Layer using Web MVC module

=====

Spring REST

=====

-> It is used to develop REST APIs (distributed applications)

-> To communicate from one app to another app we will use REST APIs.

=====

Spring Security

=====

-> It is used to add Security for our applications

=====

Spring Batch

=====

-> It is used to develop Robust batch applications

Batch Operation => Bulk Operation

Ex: Loading excel sheet data into Database / Export DB data into Excel sheet

Generate & send bank acc statement

Generate & send post paid mobile bill

Generate & send credit card bill

=====
Spring Cloud
=====

-> It is used to develop application with Microservices Architecture

- 1) Service Registry
- 2) API Gateway
- 3) FeignClient etc...

=====
What Type of applications we can develop using Spring ?
=====

- 1) Stand-Alone applications
- 2) Web Applications
- 3) Distributed Applications
- 4) Batch Applications

=> The application which can be accessed only in one system is called as Standalone application.

Ex: Notepad, Eclipse, SQL-Developer, Calc....

=> The application which can be accessed by multiple users at a time is called as Web Application.

Ex: Gmail, naukri, facebook, youtube etc...

Note: Web Applications are used for Customer to Business communication (C 2 B).

=> The application which is communicating with another application is called as Distributed Application.

Passport App <-----> AADHAR App

MakeTrip App <-----> IRCTC App

Swiggy <-----> Gpay / Phonepay

Frontend App <-----> Backend App

Note: Distributed Applications are used for Business to Business communication (B 2 B).

=> The application which performs bulk operation is called as Batch Application.

- 1) Generate & Send Bank Acc statement to Acc holders
- 2) Generate & Send Post Paid Bills to customers
- 3) Crediting Salaries to all govt employees

=====
Is Spring replacing Java and J2EE ?
=====

No, Spring is internally using Java and J2EE. We can say spring is extension for Java and J2EE.

=====
Environment Setup
=====

- 1) Download and Install JDK 17

Link :

https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.msi

- 2) Set JAVA_HOME and Path for Java

JAVA_HOME = C:\Program Files\Java\jdk-17

Path = C:\Program Files\Java\jdk-17\bin

- 3) Download and install IDE (Ex: STS / IntelliJ IDEA)

IntelliJ Download Link :

<https://www.jetbrains.com/idea/download/?section=windows>

STS Download Link : <https://spring.io.xy2401.com/tools3/sts/all/>

=====
What is Spring Boot ?
=====

=> Spring Boot is extension for Spring Framework.

=> Spring Boot is one approach to develop spring based applications.

Note: SpringBoot is internally using Spring.

=> What type of apps we can develop using Spring same type of apps can be developed using Spring Boot.

=> Spring Boot is used for rapid application development.

Spring Framework ==> Manual Configurations

Spring Boot ==> Auto Configurations

=====

Spring Boot Advantages

=====

1) Pom Starters (dependencies simplified)

Ex: web-starter, data-jpa-starter, security-starter, mail-starter

2) AutoConfiguration

Ex: creating connection pool, deploy web app to tomcat server

3) Embedded Servers

Ex: Tomcat, Jetty and Netty

4) Actuators (Monitor and manage our app)

Ex: health, threads-info, heap-info, config-props, url-mappings, beans...

=====

What is Auto Configuration ?

=====

=> It is the process of identifying configurations required for the application and provide those configurations in runtime of application.

=> Spring Boot will provide Auto configuration based on dependencies added in project.

web-starter : start tomcat server + deploy app in tomcat server

security-starter : It will secure our application with login page

=====

Developing First Boot Application

=====

=> To create boot applications we will use "start.spring.io" website (Spring Initializr website)

Project : Maven
Language : Java
Spring Boot : 3.1.2
Group : in.ashokit
Artifact : App1
Package Name : in.ashokit
Packaging : jar
Java : 17

=> Download project as zip file & extract zip file

=> Import project into IntelliJ IDE / STS IDE.

=====

Spring Boot Project Folder Structure

=====

src/main/java : To keep project source code (.java files)

- Application.java (Spring Boot Application Start Class)

src/main/resources : To keep project resources (.yaml, .properties, .sql files)

- application.properties / application.yaml

src/test/java : To keep project Junit classes

- ApplicationTest.java

Maven Dependencies : Dependencies downloaded will display here

target : .class files will be created here

pom.xml : Project Object Model (maven configuration files)

=====

What is IOC Container

=====

=> IOC stands for Inversion of control

=> IOC is a principle which is used to manage and collaborate dependencies among the objects in the application.

=> IOC is responsible to perform Dependency Injection (DI).

=====

What is Dependency Injection ?

=====

=> Injecting one class object into another class object is called as DI.

=> IOC can perform DI in 3 ways

- 1) Setter Injection (SI)
- 2) Constructor Injection (CI)
- 3) Field Injection (FI)

=> To perform Dependency Injection we will use @Autowired annotation

Note: Using IOC & DI we can develop our classes with loosely coupling.

=====

What is Spring Bean ?

=====

=> The java class which is managed by IOC is called as Spring Bean.

=> Spring Beans lifecycle will be managed by IOC.

=====

How to represent Java class as Spring Bean ?

=====

=> Using below annotations we can represent normal java class as Spring Bean

- 1) @Component
- 2) @Service
- 3) @Repository
- 4) @Configuration & @Bean
- 5) @Controller

6) @RestController

=====
What is Component Scanning ?
=====

- => It is the process of identifying spring beans available in the project
- => Component Scanning will start from base package name
- => The package which contains start class is called as base-package
- => After base-pkg scanning completed then it will scan sub-packages of base package.

```
in.ashokit -----> base package
in.ashokit.beans ---> will be scanned
in.ashokit.dao ----> will be scanned
in.ashokit.service ---> will be scanned
com.ashokit.util ----> will not be scanned
```

=====
Q) Can we specify more than one base package in project ?
=====

Ans) Yes we can do it using @ComponentScan annotation at start class.

```
@SpringBootApplication
@ComponentScan(basePackages = {"in.ashokit", "com.tcs"})
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

=====
How to get Bean Object from IOC Container ?
=====

- => using getBean() method we can get bean object from IOC container

=> We can get IOC obj using run method ()

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        ConfigurableApplicationContext context =
            SpringApplication.run(Application.class, args);

        Engine engine = context.getBean(Engine.class);
        engine.start();

        Car car = context.getBean(Car.class);
        car.drive();

    }
}
```

=====

Spring Bean Scopes

=====

=> Bean scope will represent how many objects should be created for Spring Bean class.

=> We have below 4 scopes in spring

- 1) singleton (default)
- 2) prototype
- 3) request
- 4) session

-> Singleton means only one object will be created for our bean class. If we don't specify any scope then default scope will be singleton.

-> Prototype means everytime new object will be created.

-> request and session scopes will be used in Spring Web MVC.

=> To define Scope of the bean we will use @Scope annotation

```
@Component
@Scope("prototype")
public class Engine {

    public Engine() {
        System.out.println("Engine - Constructor Executed");
    }
}
```

```

        public void start() {
            System.out.println("Engine Starting....!!");
        }
    }
}

```

```

=====
=====

```

Requirement : Develop one application using Engine and Car.

Car class drive () method should call Engine class start () method.

```

=====
=====

```

```

@Component
public class Engine {

    public Engine() {
        System.out.println("Engine - Constructor Executed");
    }

    public int start() {
        System.out.println("Engine Starting....!!");
        return 1;
    }
}

```

```

-----
@Component
public class Car {

    @Autowired
    private Engine eng;

    public Car() {
        System.out.println("Car :: Constructor Executed");
    }

    public void drive() {
        int start = eng.start();
        if (start >= 1) {
            System.out.println("Journey Started....");
        }
    }
}

```

```

-----
@SpringBootApplication
public class Application {

    public static void main(String[] args) {

```



```

        ConfigurableApplicationContext context =
            SpringApplication.run(Application.class, args);

        Car car = context.getBean(Car.class);
        car.drive();
    }
}

```

=====

What is Dependency Injection ?

=====

=> Injecting one class object into another class object is called as DI.

=> IOC can perform DI in 3 ways

- 1) Setter Injection (SI)
- 2) Constructor Injection (CI)
- 3) Field Injection (FI)

=> To perform Dependency Injection we will use @Autowired annotation

Note: Using IOC & DI we can develop our classes with loosely coupling.

=====

What is Setter Injection ?

=====

=> Injecting dependent object into target object through target class setter method is called as Setter Injection.

```

@Component
public class Car {

    private Engine eng;

    @Autowired
    public void setEng(Engine eng){
        this.eng = eng;
    }

}

```

=====

What is Constructor Injection ?

=====

=> Injecting Dependent object into target object using target class constructor is called as Constructor Injection.

```
@Component
public class Car {

    private Engine eng;

    public Car(Engine eng){
        this.eng = eng;
        System.out.println("Car :: Constructor");
    }

}
```

Note: When we have only one parameterized constructor in target class like above then @Autowired annotation is optional

=> If we have two constructors like below then we have to specify @Autowired annotation at constructor to inform IOC to call that constructor to create object.

```
@Component
public class Car {

    private Engine eng;

    public Car(){
        System.out.println("Car :: 0 - Constructor");
    }

    @Autowired
    public Car(Engine eng){
        this.eng = eng;
        System.out.println("Car :: Param - Constructor");
    }

}
```

=====
What is Field Injection ?
=====

=> Injecting Dependent object into target object using target class variable/field is called as Field Injection.

```
public class Car {
```

```
@Autowired
private Engine eng;

}
```

Note: Field Injection internally uses Reflection api to initialize private variables.

- 1) What is Spring Bean ?
- 2) How to represent Java class as Spring Bean ?
- 3) What is IOC Container ?
- 4) How to get bean object from IOC
- 5) What is Dependency Injection ?
- 6) What is Setter Injection ?
- 7) What is Constructor Injection ?
- 8) What is Field Injection ?
- 9) What is @Autowired
- 10) What is Bean Scope

@Component : It is used to represent java class as Spring Bean.

@Service : It is used to represent java class as Spring Bean.

@Repository : It is used to represent java class Spring Bean.

@Controller : It is used to represent java class Spring Bean (C2B).

@RestController : It is used to represent java class Spring Bean (B2B).

@Configuration : It is used to represent java class as Configuration class. It is used to represent @Bean methods to customize object creation.

Note: @Bean is method level annotation.

```
@Configuration
public class AppConfig {

    public AppConfig(){
        System.out.println("AppConfig :: Constructor");
    }

    @Bean
    public PwdUtils createPwdUtil(){
        System.out.println("createPwdUtil() method called...");
        PwdUtils pw = new PwdUtils();
        //logic;
        return pw;
    }
}
```

=====
What is Autowiring ?
=====

=> It is used to inform IOC container to inject dependent object into target object.

=> To enable Autowiring we will use @Autowired

=> We can use @Autowired at 3 places

- 1) setter method level
- 2) constructor level
- 3) field level

=> Autowiring is having 2 modes

- 1) byName
- 2) byType

=> byName means it will identify dependent bean object based on bean name

=> byType means it will identify dependent bean object based on data type

Note: When data type is interface then it will check for implementation classes.

=> If we have 2 implementation classes for interface then we will get Ambiguity

problem, application can't start.

=> To resolve ambiguity in Autowiring we have 2 options

- 1) @Qualifier => To specify bean name for DI
- 2) @Primary => To make specific bean as Auto Wire Candidate

=====
Spring Bean Life Cycle
=====

=> Java Class which is managing by IOC is called as Spring Bean

=> Spring Bean Life Cycle will be taken care by IOC container

=> If we want to perform some init and destroy operations on the bean then we can use Life Cycle methods

init () - > after initializing bean execute some logic

destroy () -> before destroying the bean execute some logic

=> To implement bean life cycle methods we have to implement below 2 interfaces

- 1) InitializingBean(I) -> afterPropertiesSet ()
- 2) DisposableBean (I) -> destroy ()

Assignment-1
#####

Develop Spring Boot application to read csv file data (books info) and insert data into database table using plain jdbc with Layered Architecture.

====sample file content====
101,Java,300.00
102,Python,200.00
103,AWS,350.00
104,Angular,250.00

Git Hub Repo URL : https://github.com/ashokitschool/spring_file_reading_app.git

###

- 1) What is Spring
- 2) Spring Modules
- 3) Spring vs Spring Boot
- 4) Spring Boot Advantages
- 5) Spring Boot Project Creation
- 6) Spring Boot App Folder Structure
- 7) IOC Container
- 8) Dependency Injection
- 9) SI, CI and FI
- 10) Spring Annotations
 - @Component
 - @Service
 - @Repository
 - @Configuration
 - @Bean
- 11) Component Scanning (@ComponentScan)
- 12) Auto Wiring (@Autowired, @Qualifier & @Primary)
- 13) Bean Scopes (@Scope)
- 14) Bean Life Cycle Method
- 15) Simple Project Implementing (Read file data and insert into DB)
- 16) @Value annotation